

Teamwork — Session 6: extract Community/posts, independently

Context

Work in your team. The class splits into **six teams of four students**.

In class you extracted the Opportunities routes out of `server/index.js` into `server/routes/opportunities.js` + `server/controllers/opportunities.js`, mounted with `express.Router()`, sharing state through `server/store.js` and `server/utils.js`. Community/posts still has all four of its routes written directly inline in `server/index.js`.

Goal

Independently repeat the exact same extraction, for a different, real resource this app already has: Community posts.

What to build

1. `server/store.js` gains `posts` (mirroring how it already shares `opportunities`) and a small `claimNextPostId()` function, since a new post's id needs to come from somewhere every file can trust.
2. `server/controllers/posts.js`, exporting four functions, moved verbatim from the matching routes already in `server/index.js`: `getAllPosts`, `getPostById`, `createPost`, `setPostLiked`.
3. `server/routes/posts.js`, an `express.Router()` wiring those four functions to `GET /`, `GET /:id`, `POST /`, `PATCH /:id`.
4. `server/index.js` mounts it: `app.use("/api/posts", postsRouter)`, and the four old inline routes are removed.

Every request this app already makes to `/api/posts...` must behave identically once you're done — this is a pure extraction, not a new feature.

Rules

Allowed — everything from today's live coding: `express.Router()`, `app.use(mountPath, router)`, a controller file exporting plain `(req, res)` functions, and the exact `store.js` / `utils.js` sharing pattern already built for Opportunities.

Not yet (do not use these) — a database, `req.query`, any new npm package, a validation library, an Express error-handling middleware (only discussed conceptually today, never implemented), Redux, authentication, or CORS configuration.

Do not modify `GET /api/opportunities/...` or the request-logging middleware — this task only extracts Community/posts.

Test it

- `GET /api/posts` still returns the full, sorted list.
- `GET /api/posts/:id` still returns `200` for a real id and `404` for a made-up one.
- `POST /api/posts` still validates `content / category` exactly as before, returning `400` for invalid input and `201` with the new post on success.
- `PATCH /api/posts/:id` still toggles `liked / likes` exactly as before.
- `node --check index.js` (from `server/`) reports no syntax errors.

Done when

- All four Community/posts routes are served by `routes/posts.js` + `controllers/posts.js`, not inline in `index.js`.
- Every one of the four behaviours above is actually tested with a real tool (curl/Postman/Thunder Client), not just read from the code.
- `node --check index.js` passes.
- Your team's attempt is committed and pushed to your own branch.

Working time

About **25 minutes**, then the official solution is demonstrated.

Before the solution is shown

Commit and push your team's attempt to your own `student-<your-github-username>` branch **before** the official solution is demonstrated. Pushing your own attempt first is part of the exercise.

Be ready to answer

- Why does `posts` need a `claimNextPostId()` function in `store.js`, when `opportunities` never needed anything like it?
- What would happen if `controllers/posts.js` imported `initialPosts` directly from `data.js` instead of `posts` from `store.js`?
- Could `index.js`'s health check still report an accurate `posts.length` after this extraction? Why or why not?